

01 / AUTONOMOUS ROBOTICS

Basic Line Follower Robot

| Closed-Loop Infrared Tracking with ESP32 & TB6612FNG Driver.

ACTUATOR 26

FIELD NOTES / 2026

02 / SYSTEM ARCHITECTURE

4 Core Components of a Line Follower

The robot operates a continuous sense-decide-act feedback loop:

BRAIN**ESP32 Microcontroller**

Executes control logic at high speeds, reading digital IR sensor inputs and generating PWM motor signals.

PERCEPTION**Dual IR Line Sensors**

Infrared emitter/receiver pairs placed on the underside to detect light vs dark surface boundaries.

ACTUATION**TB6612FNG Driver**

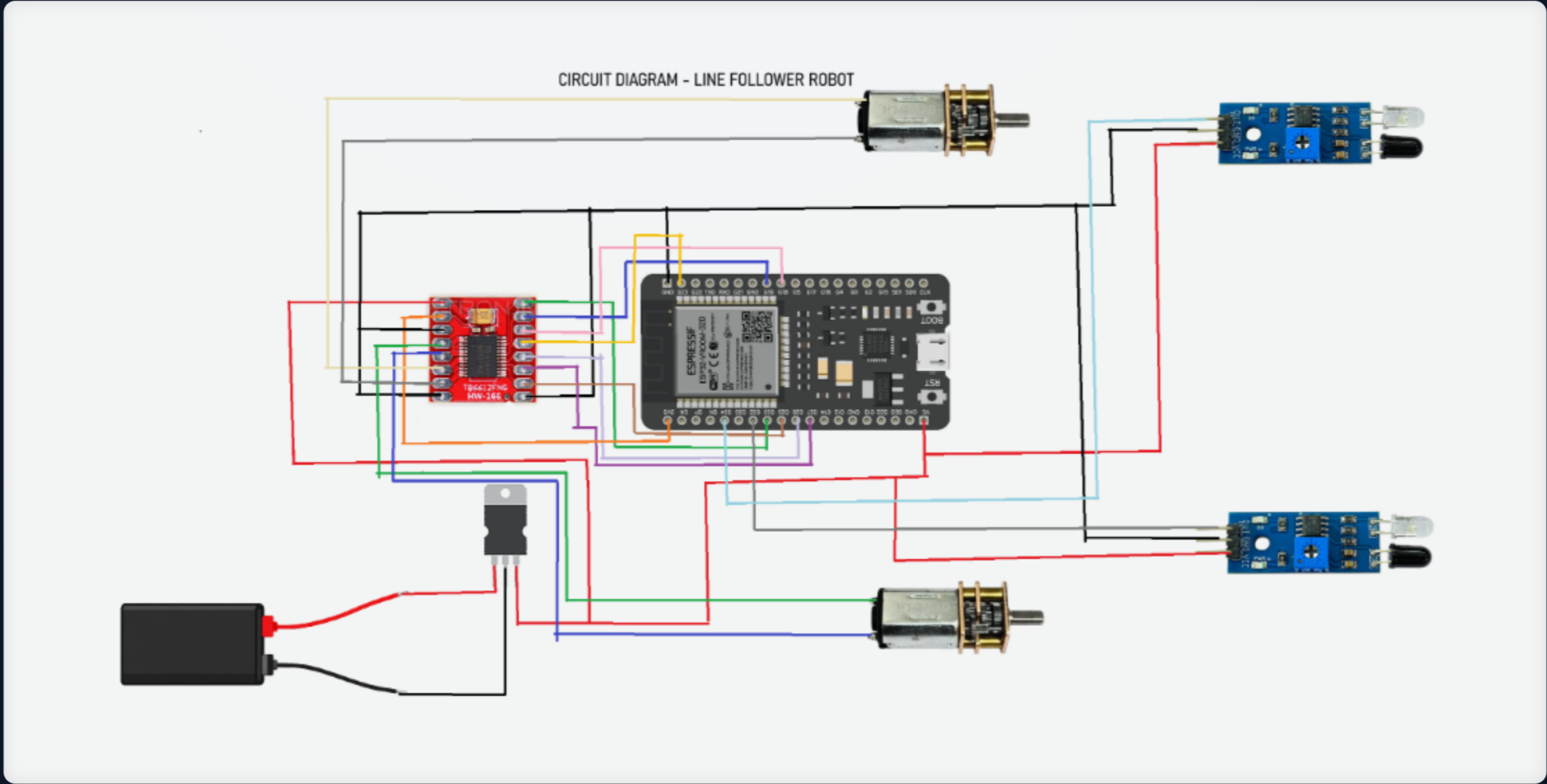
High-efficiency MOSFET dual H-Bridge driving two DC gearmotors with independent speed and direction.

POWER**LiPo / Battery Supply**

Provides separate motor power (VM) and regulated logic voltage to the ESP32.

03 / SCHEMATIC DIAGRAM

Circuit Diagram & Hardware Interconnects



ESP32 MCU
3.3V Logic

TB6612FNG Driver
Dual H-Bridge

Dual IR Sensors
GPIO 32 & 34

Common GND
Unified Ground

04 / SENSOR PRINCIPLE

Infrared Reflectance Light vs Dark Detection

Understanding how IR reflectance sensors differentiate surface boundaries:

1. White Surface (Reflective)

White surfaces reflect infrared light back to the phototransistor photodiode.

DIGITAL OUTPUT

LOW (0V Signal)

Phototransistor conducts -> Signal pin drops LOW.

2. Black Line (Absorptive)

Black tape absorbs infrared light, preventing reflection to the receiver.

DIGITAL OUTPUT

HIGH (3.3V Signal)

Phototransistor off -> Internal pull-up pulls pin HIGH.

ESP32 Pinout Assignment

Line Follower Config

Exact GPIO pin assignments configured in `line_follower/src/main.cpp`:

TB6612FNG DRIVER PINS

RC_MOTOR_STBY	GPIO 23 (Standby Enable)
RC_LEFT_PWM	GPIO 33 (Left Speed PWM)
RC_LEFT_IN1 / IN2	GPIO 18 / GPIO 19
RC_RIGHT_PWM	GPIO 25 (Right Speed PWM)
RC_RIGHT_IN1 / IN2	GPIO 26 / GPIO 27

IR SENSORS & SPEED

RC_IR_LEFT	GPIO 32 (Left IR Input)
RC_IR_RIGHT	GPIO 34 (Right IR Input)
SPEED	100 (PWM Duty Cycle)
BAUD RATE	115200 Baud



06 / CONTROL ALGORITHM

Navigation Truth Table

4-State Steering Matrix

Sensors detect line alignment to determine steering action:

LEFT IR (PIN 32)	RIGHT IR (PIN 34)	LEFT MOTOR STATE	RIGHT MOTOR STATE	ROBOT ACTION
LOW (Surface)	LOW (Surface)	FORWARD (SPEED 100)	FORWARD (SPEED 100)	Move Straight Ahead
HIGH (Line)	LOW (Surface)	STOP (0 PWM)	FORWARD (SPEED 100)	Pivot Turn Left
LOW (Surface)	HIGH (Line)	FORWARD (SPEED 100)	STOP (0 PWM)	Pivot Turn Right
HIGH (Line)	HIGH (Line)	STOP (0 PWM)	STOP (0 PWM)	Stop (End of Line)

07 / CODE BREAKDOWN

Initialization & Pin Mode Setup

Configuring GPIO directions & bringing driver out of standby in setup():

```
// setup() from line_follower/src/main.cpp

void setup() {
  Serial.begin(115200);
  pinMode(RC_MOTOR_STBY, OUTPUT);
  pinMode(RC_LEFT_PWM, OUTPUT);   pinMode(RC_LEFT_IN1, OUTPUT);   pinMode(RC_LEFT_IN2, OUTPUT);
  pinMode(RC_RIGHT_PWM, OUTPUT);  pinMode(RC_RIGHT_IN1, OUTPUT);  pinMode(RC_RIGHT_IN2, OUTPUT);
  pinMode(RC_IR_LEFT, INPUT);
  pinMode(RC_IR_RIGHT, INPUT);

  digitalWrite(RC_MOTOR_STBY, HIGH); // Enable TB6612FNG Driver
}
```

08 / CODE BREAKDOWN

Sensor Sampling & Telemetry Logging

Reading digital IR levels and logging debug statements to Serial:

```
// Sampling IR Sensor Inputs in loop()

void loop() {
  int leftSensorValue = digitalRead(RC_IR_LEFT);
  int rightSensorValue = digitalRead(RC_IR_RIGHT);

  Serial.print("LEFT IR: "); Serial.println(leftSensorValue);
  Serial.print("RIGHT IR: "); Serial.println(rightSensorValue);

  // Decision Logic Follows...
}
```

09 / CODE BREAKDOWN

Differential Steering Decision Tree Code

Executing directional movement based on sensor conditions:

```
if (leftSensorValue == LOW && rightSensorValue == LOW) {  
    // Both on surface -> Move forward  
    digitalWrite(RC_LEFT_IN1, HIGH); digitalWrite(RC_LEFT_IN2, LOW); analogWrite(RC_LEFT_P  
    digitalWrite(RC_RIGHT_IN1, HIGH); digitalWrite(RC_RIGHT_IN2, LOW); analogWrite(RC_RIGHT_  
} else if (leftSensorValue == HIGH && rightSensorValue == LOW) {  
    // Left on line -> Turn left  
    digitalWrite(RC_LEFT_IN1, LOW); digitalWrite(RC_LEFT_IN2, LOW); analogWrite(RC_LEFT_P  
    digitalWrite(RC_RIGHT_IN1, HIGH); digitalWrite(RC_RIGHT_IN2, LOW); analogWrite(RC_RIGHT_  
} else if (leftSensorValue == LOW && rightSensorValue == HIGH) {  
    // Right on line -> Turn right  
    digitalWrite(RC_LEFT_IN1, HIGH); digitalWrite(RC_LEFT_IN2, LOW); analogWrite(RC_LEFT_P  
    digitalWrite(RC_RIGHT_IN1, LOW); digitalWrite(RC_RIGHT_IN2, LOW); analogWrite(RC_RIGHT_  
} else {  
    // Both on line -> Stop  
    digitalWrite(RC_LEFT_IN1, LOW); digitalWrite(RC_LEFT_IN2, LOW); analogWrite(RC_LEFT_P  
    digitalWrite(RC_RIGHT_IN1, LOW); digitalWrite(RC_RIGHT_IN2, LOW); analogWrite(RC_RIGHT_  
}  
delay(100);
```

10 / HARDWARE TUNING

Speed Tuning & Sensor Calibration

SPEED PARAMETER**#define SPEED 100**

Controls maximum forward PWM duty cycle. Increase for speed, decrease if robot overshoots tight curves.

HEIGHT SENSITIVITY**Sensor Height (3mm - 5mm)**

Keep IR modules 3mm to 5mm above the track surface for crisp high-contrast reflection signals.

POTENTIOMETER ADJUST**Threshold Trimmer**

Adjust potentiometer on IR modules until onboard LED toggles sharply between tape and floor.

LOOP DELAY**Sampling Rate**

A 100ms delay in loop() prevents Serial buffer overload while providing smooth steering evaluation.

11 / TROUBLESHOOTING

4 Common Line Follower Pitfalls

ISSUE 1**Ambient Light Interference**

Direct sunlight or bright overhead lights can blind IR receivers. Shield sensor array with a dark skirt.

ISSUE 2**Reversed Motor Direction**

If robot spins in circles when going forward, swap motor terminals (IN1/IN2) or swap motor wire pairs.

ISSUE 3**Battery Voltage Sag**

Weak batteries cause ESP32 micro-resets when motors start. Always use separate battery for VM rail.

ISSUE 4**STBY Pin Disconnected**

If motors don't move at all, verify GPIO 23 (STBY) is driven HIGH in setup()!



12 / READY TO TRACK

Sense. Decide. Track.

Your ESP32 Line Follower robot is fully configured for autonomous track navigation.

LINE FOLLOWER · ESP32 · ACTUATOR 26

LET'S
TRACK IT